

Практическая работа 21: Добавление ViewModel в приложение

1. Прежде чем начать

Введение Dessert Clicker работает со встроенным состоянием и данными. В этом упражнении вы удалите встроенное состояние, данные и логику из MainActivity и переместите их во ViewModel.

Абстрагирование логики приложения от представления до ViewModel - это современная практика разработки под Android. Эта практика дает следующие преимущества:

Код становится более читабельным для других разработчиков. Код становится более тестируемым. Несколько разработчиков могут одновременно работать над приложением, не мешая работе других разработчиков. Код решения доступен в конце, но постарайтесь решить упражнения, прежде чем проверять ответы. Рассматривайте решение как один из способов реализации приложения.

Предварительные условия Курс «Основы Android в Compose» через кодельную лабораторию «ViewModel и State в Compose». Что вам понадобится Компьютер с доступом в интернет и установленной Android Studio Код решения для приложения Dessert Clicker Что вы будете создавать В этих практических задачах вы улучшите архитектуру приложения Dessert Clicker, добавив ViewModel для обработки данных и логики приложения.

Практические задачи разбиты на секции, в которых вы выполните следующие шаги по отдельности:

Обновите и добавьте необходимые зависимости. Создайте класс ViewModel.

2. Загрузите стартовый код

URL стартового кода:

<https://github.com/google-developer-training/basic-android-kotlin-compose-training-dessert-clicker>

Название ветки со стартовым кодом: main

В Android Studio откройте папку basic-android-kotlin-compose-training-dessert-clicker. Откройте код приложения Dessert Clicker в Android Studio.

3. Настройка зависимостей

Добавьте следующую переменную в файл build.gradle вашего проекта:

```
buildscript {  
    ext {  
        ...  
        lifecycle_version = '2.5.1'  
    }  
}
```

```
}  
}
```

Добавьте следующую зависимость в файл app/build.gradle:

```
dependencies {  
    ...implementation "androidx.lifecycle:lifecycle-viewmodel-  
compose:$lifecycle_version"  
}
```

4. Создайте класс состояния пользовательского интерфейса

В настоящее время композит `DessertClickerApp()` в `MainActivity` содержит данные и состояние, которые управляют пользовательским интерфейсом.

Создайте класс данных, который будет содержать все необходимые данные для пользовательского интерфейса. Данные в этом классе заменят данные, которыми в настоящее время управляет `DessertClickerApp()` composable.

5. Создание ViewModel

Создайте класс `ViewModel` с помощью компонента Jetpack `ViewModel`. Вы используете `ViewModel` для управления состоянием пользовательского интерфейса.

6. Перенесите логику и данные приложения в ViewModel

Перенесите логику из `MainActivity` в `ViewModel` и сделайте данные состояния пользовательского интерфейса доступными с помощью созданного вами класса `UI state`. Удалите все данные и логику управления состоянием из `MainActivity`.

Попробуйте справиться с этой задачей самостоятельно. При необходимости обратитесь к кодебалету `ViewModel and State in Compose` за руководством.

7. Вызов ViewModel

Используйте данные и методы, которые предоставляет `ViewModel`, для управления пользовательским интерфейсом в `MainActivity`.

8. Код решения

URL-адрес начального кода:

<https://github.com/google-developer-training/basic-android-kotlin-compose-training-dessert-clicker>

Название ветки с начальным кодом: viewmodel